Ejercicios de aplicación 🤓

L4: Redes neuronales convolutivas

Es hora de que pongas en práctica todo lo aprendido. 🤓
Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que, a continuación, encontrarás diferentes desafíos que podrás resolver de forma independiente y a tu ritmo.

Más adelante, conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío 🎯**

Diseñar, implementar y entrenar una red neuronal convolutiva (CNN) en Python para clasificar imágenes del dataset MNIST (dígitos manuscritos).

<u>Objetivo</u>
Aplicar los conceptos aprendidos sobre redes convolutivas (convolución, ReLU, pooling, flattening y fully connected) implementando una arquitectura simple con Keras. El modelo deberá ser capaz de reconocer los números del 0 al 9 en imágenes de 28x28 píxeles.

<u>Pasos sugeridos</u>

- *Cargar el dataset MNIST desde Keras.*
- *Normalizar las imágenes y transformar las etiquetas en formato categórico.*
- *Construir una red convolutiva con, al menos, lo siguiente:*
    - 2 capas Conv2D + ReLU
    - 2 capas MaxPooling2D
    - 1 capa Flatten y 1 Dense final con activación softmax
- *Compilar y entrenar el modelo.*

- *Evaluar el rendimiento del modelo en el conjunto de prueba.*
- *Mostrar una predicción con una imagen real del test set.*

2. **¿Dónde se lleva a cabo?** 👩‍💻

   Visual Studio Code

3. **Tiempo de dedicación** ⏳

   2 horas

4. **Recursos** 🛠️

   Dataset MNIST incluido en Keras: from tensorflow.keras.datasets import mnist

   https://keras.io/examples/vision/mnist_convnet/

5. **Plus** ➕
   - *Agrega Dropout para reducir el sobreajuste.*
   - *Prueba con más capas convolutivas y observa si mejora la precisión.*
   - *Grafica una matriz de confusión con sklearn.metrics.confusion_matrix.*

6. ⚠️**Condición**

   Esta práctica o ejercitación no requiere ser entregada o evaluada por el mentor. No obstante, puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío 👇

L4: Redes neuronales convolutivas

```python
# Paso 1: Importar librerías
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

# Paso 2: Cargar dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Paso 3: Preprocesar datos
X_train = X_train.reshape(-1, 28, 28, 1).astype("float32") / 255.0
X_test = X_test.reshape(-1, 28, 28, 1).astype("float32") / 255.0
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

# Paso 4: Construir modelo
model = Sequential()
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))  # regularización opcional
model.add(Dense(10, activation='softmax'))  # 10 clases

# Paso 5: Compilar y entrenar
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=5, validation_split=0.1)

# Paso 6: Evaluar
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f'Precisión en el conjunto de prueba: {test_accuracy:.2%}')
```

Paso n.° 7: visualizar predicciones

```
# Predecir un número aleatorio
random_index = np.random.randint(0, X_test.shape[0])
img = X_test[random_index]
prediction = model.predict(np.expand_dims(img, axis=0))
predicted_label = np.argmax(prediction)

plt.imshow(img.reshape(28,28), cmap='gray')
plt.title(f'Predicción: {predicted_label}')
plt.axis('off')
plt.show()
```